Lectio — Final Build Plan

Project: Lectio — Educational Component Library **Stack:** SvelteKit + shadcn-svelte + TypeScript + Tailwind **Goal today:** Prove the architecture. Five components, one template, working showcase. Nothing more.

What Lectio Is

A community-maintainable educational component library.
Components are cognitive interactions — not visual shapes.
Templates are instructional strategies built from components.
Anyone can contribute a template. Components are governed.

The File Structure

```
lectio/
├── src/
|   ├── lib/
|   |   ├── types.ts              ← content schemas (write first)
|   |   ├── registry.ts           ← component metadata (write second)
|   |   ├── validate.ts           ← capacity warnings (write third)
|   |   └── dummy-content.ts      ← calculus test content (write fourth)
|   |
|   ├── components/
|   |   ├── ui/                   ← shadcn-svelte auto-generated
|   |   └── lectio/
|   |       ├── HookHero.svelte
|   |       ├── ExplanationBlock.svelte
|   |       ├── DefinitionCard.svelte
|   |       ├── WorkedExampleCard.svelte
|   |       ├── PracticeStack.svelte
|   |       ├── PitfallAlert.svelte
|   |       ├── GlossaryRail.svelte     ← if time allows
|   |       ├── WhatNextBridge.svelte   ← if time allows
|   |       └── index.ts               ← exports all
|   |
|   ├── templates/
|   |   └── GuidedConceptPath.svelte
|   |
|   └── routes/
```

```
|       ├── +layout.svelte            ← sidebar navigation
|       ├── +page.svelte              ← landing page
|       ├── components/
|       |   └── +page.svelte          ← component showcase
|       └── templates/
|           └── +page.svelte          ← template showcase
|
└── package.json
```

Step 1 — Create the project (10 minutes)

```bash
npx sv create lectio
# Choose: SvelteKit minimal, TypeScript, Tailwind

cd lectio

# Install shadcn-svelte
npx shadcn-svelte@latest init

# Install the primitives you need
npx shadcn-svelte@latest add accordion
npx shadcn-svelte@latest add collapsible
npx shadcn-svelte@latest add card
npx shadcn-svelte@latest add dialog
npx shadcn-svelte@latest add alert
npx shadcn-svelte@latest add badge
npx shadcn-svelte@latest add button
npx shadcn-svelte@latest add separator
npx shadcn-svelte@latest add scroll-area

# Supporting libraries
npm install lucide-svelte
npm install katex
npm install @types/katex
```

Step 2 — The four foundation files (30 minutes)

Write these before any component. Everything depends on them.

src/lib/types.ts

```typescript
```

```typescript
// Content schemas — what the LLM fills in,
// what components render, what templates assemble

export interface HookHeroContent {
  headline: string    // max 12 words
  body: string        // max 80 words
  anchor: string      // the felt need this creates
}

export interface ExplanationContent {
  body: string        // max 350 words
  emphasis: string[]  // max 3 key phrases to bold inline
}

export interface DefinitionContent {
  term: string
  formal: string      // max 80 words
  plain: string       // max 60 words
  etymology?: string
}

export interface WorkedStep {
  label: string       // max 12 words — why this step
  content: string     // max 80 words — what was done
}

export interface WorkedExampleContent {
  title: string
  setup: string       // max 60 words
  steps: WorkedStep[] // max 6
  conclusion: string  // max 40 words
}

export interface PracticeProblem {
  difficulty: 'warm' | 'medium' | 'cold'
  question: string    // max 100 words
  hint: string        // max 60 words
  writein_lines?: number // 0-8 for print
}

export interface PracticeContent {
  // Tuple enforces exactly 3 problems
  problems: [PracticeProblem, PracticeProblem, PracticeProblem]
```

```
}

export interface GlossaryTerm {
  term: string
  definition: string  // max 30 words
  used_in?: string
}

export interface GlossaryContent {
  terms: GlossaryTerm[] // max 8
}

export interface PitfallContent {
  misconception: string  // max 20 words
  correction: string     // max 80 words
  example?: string       // max 40 words
}

export interface WhatNextContent {
  body: string   // max 50 words
  next: string   // max 15 words
}

// The full section — what drives a complete template render
export interface SectionContent {
  section_id: string
  title: string
  subtitle?: string
  subject: string
  grade_band: string
  template_id: string
  hook: HookHeroContent
  explanation: ExplanationContent
  definition?: DefinitionContent
  worked_example?: WorkedExampleContent
  pitfall?: PitfallContent
  practice: PracticeContent
  glossary?: GlossaryContent
  what_next: WhatNextContent
}
```

---

src/lib/registry.ts

typescript

```typescript
// The component registry — single source of truth
// Drives the showcase page, template generation,
// and contribution review checklist

export interface ComponentMeta {
  id: string
  name: string
  purpose: string
  cognitiveJob: string
  subjects: string[]
  behaviourModes: string[]
  shadcnPrimitive: string
  capacity: Record<string, number | string>
  printFallback: string
}

export const componentRegistry: Record<string, ComponentMeta> = {
  HookHero: {
    id: 'hook-hero',
    name: 'HookHero',
    purpose: 'Opens a section by creating felt need before explanation',
    cognitiveJob: 'Create felt need',
    subjects: ['universal'],
    behaviourModes: ['static', 'fade-in'],
    shadcnPrimitive: 'Card (none — pure layout)',
    capacity: {
      headlineMaxWords: 12,
      bodyMaxWords: 80,
    },
    printFallback: 'Pull quote block with left border',
  },

  ExplanationBlock: {
    id: 'explanation-block',
    name: 'ExplanationBlock',
    purpose: 'Sustained prose that builds a mental model of the concept',
    cognitiveJob: 'Build understanding through prose',
    subjects: ['universal'],
    behaviourModes: ['static', 'progressive-reveal'],
    shadcnPrimitive: 'Typography',
    capacity: {
      bodyMaxWords: 350,
      emphasisMaxItems: 3,
```

```
    },
    printFallback: 'Static prose, emphasis preserved',
  },

  DefinitionCard: {
    id: 'definition-card',
    name: 'DefinitionCard',
    purpose: 'Anchors a formal term with both formal and plain versions',
    cognitiveJob: 'Anchor formal knowledge',
    subjects: ['universal'],
    behaviourModes: ['static', 'plain-formal-toggle'],
    shadcnPrimitive: 'Card + Collapsible',
    capacity: {
      formalMaxWords: 80,
      plainMaxWords: 60,
    },
    printFallback: 'Both formal and plain shown, toggle removed',
  },

  WorkedExampleCard: {
    id: 'worked-example-card',
    name: 'WorkedExampleCard',
    purpose: 'Shows reasoning in action, step by step, each step justified',
    cognitiveJob: 'Watch reasoning in action',
    subjects: ['universal'],
    behaviourModes: ['static', 'step-reveal', 'accordion'],
    shadcnPrimitive: 'Card + Collapsible',
    capacity: {
      stepsMax: 6,
      stepsWarning: 4,
      stepLabelMaxWords: 12,
      stepContentMaxWords: 80,
      setupMaxWords: 60,
      conclusionMaxWords: 40,
    },
    printFallback: 'All steps expanded and visible',
  },

  PracticeStack: {
    id: 'practice-stack',
    name: 'PracticeStack',
    purpose: 'Three problems at calibrated difficulty, hints hidden by default',
    cognitiveJob: 'Apply understanding under calibrated difficulty',
    subjects: ['universal'],
```

```
    behaviourModes: ['hint-toggle', 'accordion-cards', 'flat-list'],
    shadcnPrimitive: 'Accordion + Collapsible',
    capacity: {
      problemsExact: 3,
      questionMaxWords: 100,
      hintMaxWords: 60,
      writeinLinesMax: 8,
    },
    printFallback: 'All problems visible, hints visible, write-in lines rendered',
  },

  PitfallAlert: {
    id: 'pitfall-alert',
    name: 'PitfallAlert',
    purpose: 'Names a specific misconception explicitly before the learner makes it'
    cognitiveJob: 'Inoculate against error',
    subjects: ['universal'],
    behaviourModes: ['static', 'expand-example'],
    shadcnPrimitive: 'Alert + Collapsible',
    capacity: {
      misconceptionMaxWords: 20,
      correctionMaxWords: 80,
      exampleMaxWords: 40,
    },
    printFallback: 'Full static, amber left border',
  },

  GlossaryRail: {
    id: 'glossary-rail',
    name: 'GlossaryRail',
    purpose: 'Vocabulary terms visible in peripheral field, updates by section',
    cognitiveJob: 'Retrieve meaning without losing place',
    subjects: ['universal'],
    behaviourModes: ['sticky-sidebar', 'drawer', 'inline-strip'],
    shadcnPrimitive: 'Card + ScrollArea',
    capacity: {
      termsMax: 8,
      termsWarning: 6,
      definitionMaxWords: 30,
    },
    printFallback: 'Inline vocabulary strip at section end',
  },

  WhatNextBridge: {
```

```
    id: 'what-next-bridge',
    name: 'WhatNextBridge',
    purpose: 'Connects the section forward — what this concept enables',
    cognitiveJob: 'Connect forward',
    subjects: ['universal'],
    behaviourModes: ['static'],
    shadcnPrimitive: 'Card (pure layout)',
    capacity: {
      bodyMaxWords: 50,
      nextMaxWords: 15,
    },
    printFallback: 'Static, amber left rule',
  },
}

// Helper — get all component ids for a given subject
export function getComponentsForSubject(subject: string): ComponentMeta[] {
  return Object.values(componentRegistry).filter(c =>
    c.subjects.includes('universal') || c.subjects.includes(subject)
  )
}
```

---

src/lib/validate.ts

```typescript
```

```
// Capacity warnings — not hard errors, just console warnings
// Prevents content from quietly overflowing components

import type {
  SectionContent, WorkedExampleContent,
  PracticeContent, GlossaryContent
} from './types'

function countWords(text: string): number {
  return text.trim().split(/\s+/).length
}

export function validateSection(section: SectionContent): string[] {
  const warnings: string[] = []

  // Hook
  if (countWords(section.hook.headline) > 12)
    warnings.push(`HookHero: headline exceeds 12 words (${countWords(section.hook.he
  if (countWords(section.hook.body) > 80)
    warnings.push(`HookHero: body exceeds 80 words (${countWords(section.hook.body)}

  // Explanation
  if (countWords(section.explanation.body) > 350)
    warnings.push(`ExplanationBlock: body exceeds 350 words`)
  if (section.explanation.emphasis.length > 3)
    warnings.push(`ExplanationBlock: more than 3 emphasis items`)

  // Definition
  if (section.definition) {
    if (countWords(section.definition.formal) > 80)
      warnings.push(`DefinitionCard: formal definition exceeds 80 words`)
    if (countWords(section.definition.plain) > 60)
      warnings.push(`DefinitionCard: plain definition exceeds 60 words`)
  }

  // Worked example
  if (section.worked_example) {
    const ex = section.worked_example
    if (ex.steps.length > 6)
      warnings.push(`WorkedExampleCard: ${ex.steps.length} steps — max is 6`)
    else if (ex.steps.length > 4)
      warnings.push(`WorkedExampleCard: ${ex.steps.length} steps — consider trimming
    ex.steps.forEach((step, i) => {
```

```
      if (countWords(step.label) > 12)
        warnings.push(`WorkedExampleCard step ${i+1}: label exceeds 12 words`)
      if (countWords(step.content) > 80)
        warnings.push(`WorkedExampleCard step ${i+1}: content exceeds 80 words`)
    })
  }

  // Practice
  section.practice.problems.forEach((p, i) => {
    if (countWords(p.question) > 100)
      warnings.push(`PracticeStack problem ${i+1}: question exceeds 100 words`)
    if (countWords(p.hint) > 60)
      warnings.push(`PracticeStack problem ${i+1}: hint exceeds 60 words`)
  })

  // Glossary
  if (section.glossary) {
    if (section.glossary.terms.length > 8)
      warnings.push(`GlossaryRail: ${section.glossary.terms.length} terms — max is 8
    else if (section.glossary.terms.length > 6)
      warnings.push(`GlossaryRail: ${section.glossary.terms.length} terms — consider
    section.glossary.terms.forEach((t, i) => {
      if (countWords(t.definition) > 30)
        warnings.push(`GlossaryRail term ${i+1}: definition exceeds 30 words`)
    })
  }

  return warnings
}

// Call this in dev — shows warnings in console
export function warnIfInvalid(section: SectionContent): void {
  if (process.env.NODE_ENV !== 'development') return
  const warnings = validateSection(section)
  warnings.forEach(w => console.warn(`[Lectio] ${w}`))
}
```

---

src/lib/dummy-content.ts

The calculus section content. Powers every component preview and the full template showcase.

typescript

```
import type { SectionContent } from './types'

export const calculusSection: SectionContent = {
  section_id: 'calc-01',
  title: 'Why does calculus exist?',
  subtitle: 'Two questions algebra cannot answer',
  subject: 'Mathematics',
  grade_band: 'secondary',
  template_id: 'guided_concept_path_v1',

  hook: {
    headline: 'How fast is something moving at this exact instant?',
    body: 'You can measure where a ball is at two moments and calculate how far it m
    anchor: 'the gap between average speed and instantaneous speed'
  },

  explanation: {
    body: 'This question forced mathematicians to invent an entirely new branch of m
    emphasis: ['derivatives', 'integrals', 'opposites of each other']
  },

  definition: {
    term: 'Calculus',
    formal: 'The mathematical study of continuous change, consisting of differential
    plain: 'A branch of mathematics built to answer two questions: how fast is somet
    etymology: 'From Latin calculus — a small stone used for counting.'
  },

  worked_example: {
    title: 'Finding instantaneous speed from a position function',
    setup: "A ball dropped from a 100-foot building has height h(t) = 100 - 16t² at
    steps: [
      { label: 'Find the position at t = 2', content: 'h(2) = 100 - 16(4) = 36 feet.
      { label: 'Calculate average speed over one second', content: 'h(3) = -44 feet.
      { label: 'Shrink the interval', content: 'Over 0.1 seconds the average is -65.
      { label: 'The limit gives the exact answer', content: 'As the interval approac
    ],
    conclusion: 'The instantaneous speed at t = 2 is exactly -64 ft/s — found by tak
  },

  pitfall: {
    misconception: 'Calculus is just faster algebra for the same problems',
    correction: 'Calculus solves problems algebra genuinely cannot. Instantaneous sp
```

```
    example: 'Any algebraic attempt to find instantaneous speed requires dividing by
  },

  practice: {
    problems: [
      {
        difficulty: 'warm',
        question: 'A car is at mile marker 10 at 1 PM, mile marker 35 at 2 PM, and m
        hint: 'Average speed = distance ÷ time. Calculate separately for 1–2 PM and
        writein_lines: 4
      },
      {
        difficulty: 'medium',
        question: 'The height of a ball thrown upward is h(t) = 40t − 5t². Calculate
        hint: 'Use (h(b) − h(a)) ÷ (b − a) for each interval and notice what happens
        writein_lines: 5
      },
      {
        difficulty: 'cold',
        question: 'Explain without calculus why it is impossible to find the speed o
        hint: 'Speed requires distance divided by time. What happens to the time val
        writein_lines: 6
      }
    ]
  },

  glossary: {
    terms: [
      { term: 'Derivative', definition: 'The instantaneous rate of change of a funct
      { term: 'Integral', definition: 'The total accumulated change — the area under
      { term: 'Limit', definition: 'What a value approaches as a variable gets infin
      { term: 'Instantaneous', definition: 'At a single exact moment, not averaged o
    ]
  },

  what_next: {
    body: 'Now that we understand why calculus exists, we need the precise tool that
    next: 'Section 2 — Limits: The Foundation of Calculus'
  }
}
```

Step 3 — Build the five components (2 hours)

Build in this order. Each one proves something.

```
1. PracticeStack      — proves shadcn Accordion + Collapsible integration
2. WorkedExampleCard  — proves step-reveal behaviour pattern
3. DefinitionCard     — proves plain/formal toggle
4. HookHero           — proves pure layout component
5. PitfallAlert       — proves shadcn Alert integration

If time allows:
6. ExplanationBlock   — proves simple prose component
7. GlossaryRail       — proves ScrollArea + sticky behaviour
8. WhatNextBridge     — proves bridge component
```

Each component:

- Lives in `src/components/lectio/`
- Imports its type from `$lib/types`
- Uses shadcn-svelte primitives underneath
- Has no dependencies on the main app

Step 4 — Assemble the template (30 minutes)

```svelte
```

```
<!-- src/templates/GuidedConceptPath.svelte -->
<script lang="ts">
  import type { SectionContent } from '$lib/types'
  import { warnIfInvalid } from '$lib/validate'
  import HookHero from '$components/lectio/HookHero.svelte'
  import ExplanationBlock from '$components/lectio/ExplanationBlock.svelte'
  import DefinitionCard from '$components/lectio/DefinitionCard.svelte'
  import WorkedExampleCard from '$components/lectio/WorkedExampleCard.svelte'
  import PracticeStack from '$components/lectio/PracticeStack.svelte'
  import PitfallAlert from '$components/lectio/PitfallAlert.svelte'
  import GlossaryRail from '$components/lectio/GlossaryRail.svelte'
  import WhatNextBridge from '$components/lectio/WhatNextBridge.svelte'

  export let section: SectionContent

  // Warn in dev if content exceeds capacity limits
  warnIfInvalid(section)
</script>

<div class="max-w-4xl mx-auto">
  <div class="lg:grid lg:grid-cols-[1fr_220px] lg:gap-8 lg:items-start">

    <!-- Main content -->
    <div class="space-y-6">

      <!-- Section header -->
      <div class="border-b pb-4">
        <div class="text-xs font-semibold tracking-widest uppercase
                    text-muted-foreground mb-1">
          {section.subject}
        </div>
        <h1 class="text-3xl font-bold tracking-tight">{section.title}</h1>
        {#if section.subtitle}
          <p class="text-muted-foreground mt-1">{section.subtitle}</p>
        {/if}
      </div>

      <!-- Hook — always first -->
      <HookHero content={section.hook} />

      <!-- Explanation -->
      <ExplanationBlock content={section.explanation} />
```

```
      <!-- Definition — if present -->
      {#if section.definition}
        <DefinitionCard content={section.definition} />
      {/if}

      <!-- Worked example — if present -->
      {#if section.worked_example}
        <WorkedExampleCard
          content={section.worked_example}
          mode="step-reveal"
        />
      {/if}

      <!-- Pitfall — if present -->
      {#if section.pitfall}
        <PitfallAlert content={section.pitfall} />
      {/if}

      <!-- Practice — always present -->
      <PracticeStack content={section.practice} />

      <!-- What's next — always last -->
      <WhatNextBridge content={section.what_next} />
    </div>

    <!-- Glossary sidebar — desktop only -->
    {#if section.glossary}
      <div class="hidden lg:block">
        <GlossaryRail content={section.glossary} />
      </div>
    {/if}

  </div>
</div>
```

Step 5 — The showcase routes (30 minutes)

src/routes/+layout.svelte

Sidebar navigation — same pattern as shadcn docs.
Links drive from the registry so they never go out of sync.

svelte

```
<script lang="ts">
  import { componentRegistry } from '$lib/registry'
</script>

<div class="flex min-h-screen">
  <aside class="w-64 border-r bg-muted/30 p-6 hidden md:block
                sticky top-0 h-screen overflow-y-auto">
    <div class="mb-8">
      <a href="/" class="text-lg font-bold tracking-tight">Lectio</a>
      <p class="text-xs text-muted-foreground mt-1">Educational UI</p>
    </div>

    <nav class="space-y-6">
      <div>
        <div class="text-xs font-semibold tracking-widest uppercase
                    text-muted-foreground mb-2">Components</div>
        <ul class="space-y-1">
          {#each Object.values(componentRegistry) as component}
            <li>
              <a href="/components#{component.id}"
                 class="block text-sm text-muted-foreground
                        hover:text-foreground py-1 transition-colors">
                {component.name}
              </a>
            </li>
          {/each}
        </ul>
      </div>

      <div>
        <div class="text-xs font-semibold tracking-widest uppercase
                    text-muted-foreground mb-2">Templates</div>
        <ul class="space-y-1">
          <li>
            <a href="/templates#guided-concept-path"
               class="block text-sm text-muted-foreground
                      hover:text-foreground py-1 transition-colors">
              Guided Concept Path
            </a>
          </li>
        </ul>
      </div>
    </nav>
```

```
  </aside>

  <main class="flex-1 min-w-0">
    <slot />
  </main>
</div>
```

src/routes/components/+page.svelte

Registry drives the component listing automatically.
Add a new component to the registry and it appears here.

```svelte
```

```
<script lang="ts">
  import { componentRegistry } from '$lib/registry'
  import { calculusSection } from '$lib/dummy-content'
  import HookHero from '$components/lectio/HookHero.svelte'
  import DefinitionCard from '$components/lectio/DefinitionCard.svelte'
  import WorkedExampleCard from '$components/lectio/WorkedExampleCard.svelte'
  import PracticeStack from '$components/lectio/PracticeStack.svelte'
  import PitfallAlert from '$components/lectio/PitfallAlert.svelte'

  // Map component ids to their preview content and component
  // Add to this as you build more components
  const previews: Record<string, any> = {
    'hook-hero':          { component: HookHero,         props: { content: calculusS
    'definition-card':    { component: DefinitionCard,    props: { content: calculus
    'worked-example-card':{ component: WorkedExampleCard, props: { content: calculus
    'practice-stack':     { component: PracticeStack,     props: { content: calculus
    'pitfall-alert':      { component: PitfallAlert,      props: { content: calculus
  }
</script>

<div class="max-w-3xl mx-auto px-6 py-16 space-y-24">
  <div>
    <h1 class="text-3xl font-bold tracking-tight mb-3">Components</h1>
    <p class="text-muted-foreground">
      Each component represents a cognitive interaction — not a visual shape.
    </p>
  </div>

  {#each Object.values(componentRegistry) as meta}
    {#if previews[meta.id]}
      <section id={meta.id} class="scroll-mt-20">
        <h2 class="text-2xl font-bold tracking-tight mb-1">{meta.name}</h2>
        <p class="text-muted-foreground mb-3">{meta.purpose}</p>

        <div class="flex gap-6 text-sm mb-6">
          <span>
            <span class="text-xs font-semibold tracking-widest uppercase
                         text-muted-foreground mr-2">Cognitive job</span>
            {meta.cognitiveJob}
          </span>
          <span>
            <span class="text-xs font-semibold tracking-widest uppercase
                         text-muted-foreground mr-2">Primitive</span>
```

```
            {meta.shadcnPrimitive}
          </span>
        </div>

        <!-- Live preview -->
        <div class="border rounded-xl p-6 bg-background mb-3">
          <svelte:component
            this={previews[meta.id].component}
            {...previews[meta.id].props}
          />
        </div>

        <p class="text-xs text-muted-foreground">
          Import from <code class="bg-muted px-1 py-0.5 rounded">@lectio/ui</code>
        </p>
      </section>
    {/if}
  {/each}
</div>
```

src/routes/templates/+page.svelte

Full template in action with real content.

```svelte
```

```
<script lang="ts">
  import GuidedConceptPath from '$templates/GuidedConceptPath.svelte'
  import { calculusSection } from '$lib/dummy-content'
</script>

<div class="px-6 py-16">
  <div class="max-w-3xl mx-auto mb-12">
    <h1 class="text-3xl font-bold tracking-tight mb-3">Templates</h1>
    <p class="text-muted-foreground leading-relaxed">
      Templates are instructional strategies built from components.
      Content below is driven by a typed schema — the same structure
      the AI generation pipeline produces.
    </p>
  </div>

  <div class="max-w-3xl mx-auto mb-6">
    <h2 class="text-xl font-semibold mb-1" id="guided-concept-path">
      Guided Concept Path
    </h2>
    <p class="text-sm text-muted-foreground">
      Hook → Explain → Define → Exemplify → Practise.
      The classic instructional arc. Predictable structure
      so learners focus on content, not navigation.
    </p>
  </div>

  <div class="border-t pt-8">
    <GuidedConceptPath section={calculusSection} />
  </div>
</div>
```

What success looks like today

By end of today these five things are true:

```
1. Educational components build cleanly on shadcn-svelte primitives
2. A template assembles from those components without coupling to the app
3. Typed content drives the entire render
4. The result looks intentional — not stitched together
5. The component registry is the single source of truth for the
   showcase page navigation — add a component to the registry
   and it appears in the docs automatically
```

If all five are true, the architecture is proven.

What not to build today

```
× Second template
× npm publishing
× Community contribution logic
× LLM-generated content
× Visual QC automation
× Advanced theming or presets
× Deployment polish
```

Today is about proving the bones.
Everything else builds on a proven foundation.

The one command to run

```bash
npm run dev
# Open http://localhost:5173
```

Three routes. Five components. One template.
Working, testable, importable.

This is the right first slice. Start with types.ts. Everything follows from the types.